ESP32 SMART HOME

System Design and Demonstration

Project Team

Trần Sỹ Nguyên	20235985
Lăng Hồng Nguyệt Anh	20235885
Lương Ngọc Vũ Long	20235967
Hà Đức Dương	20235923
Nguyễn Vũ Anh Khoa	20235957

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Presentation Outline

01 Introduction

02 Requirements

03 System Design

04 Budget

05 Progress and Demo

06 Team and Status

1. Problem and Users

The problem

In most homes, turning on a light still means walking to the switch. Even with a smart app, you still have to press the button yourself — there is no way to tell the system when the light should turn on, so it can never act automatically.

Intended users

Homeowners and renters who want everyday devices controllable from a phone, with no technical knowledge required.

Why it matters

- Control devices without going to each switch.
- See device state live, with history for monitoring.
- Automate device control through customizable rules.

1. Checkpoint 1 Scope

In scope now

- Light ON/OFF and door open/close over MQTT (`smarthome/cmd`) from the web dashboard.
- PIR motion turns the light on, auto-off after inactivity.
- RFID allowlist opens the door; unknown cards are rejected and logged.
- Live web dashboard: SSE stream of the last 300 events, device state at a glance.
- Rules engine: trigger/condition/action rules, persisted in `rules.json` across restarts.

Deliberately deferred

- Mobile app on the unified contract — alignment with the refactored topics is in progress.
- Voice control (planned after core acceptance).
- Database-backed event history; logs stay in a RAM buffer (only rules persist).
- Cloud/remote access; the demo runs on a shared LAN.
- Energy-saving measurement.

2. Functional Requirements

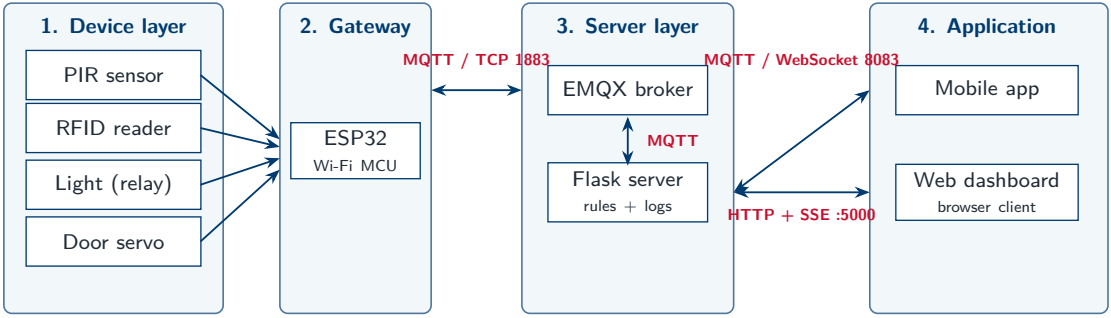
ID	What the system must do
FR1	Turn the light ON and OFF from the dashboard, with the real state confirmed back.
FR2	Open and close the model door remotely.
FR3	Turn the light on when motion is detected, and off again by itself after a configurable quiet period.
FR4	Open the door only for RFID cards on the allowlist; reject and log everyone else.
FR5	Show the live state of the light, door, motion sensor and connection on one screen.
FR6	Keep a running list of recent sensor readings, access attempts and device events.

2. Non-functional Targets

ID	Quality	What counts as passing
NFR1	Stability	Runs 30 minutes with no reset and no dropped connection.
NFR2	Mechanism	Door completes 20 open/close cycles without jamming.
NFR3	Recovery	After a broker or Wi-Fi outage, the ESP32 reconnects and resumes its state.
NFR4	Robustness	Malformed commands are rejected and logged; firmware keeps running.
NFR5	Repeatability	Every must-have scenario passes twice in a row.

Each target is checked with a short, repeatable test on the physical model — the results are shown at the demo.

3. Architecture



3. MQTT Contract per Hop

	Unified contract (firmware + server)	Mobile branch (not yet updated)
Command topic	`smarthome/cmd`, one JSON command for every device	`smarthome/cmd/light` `smarthome/cmd/door` as plain text
Feedback topics	`smarthome/logs`, one stream for states and events	`smarthome/state/+` `smarthome/event/access`
Payload	JSON for both commands and logs	Text commands, mixed JSON events
Transport	TCP 1883 for devices and the server	WebSocket 8083 for the app

Integration priority Both transports are fine — only topics and payloads have to match.

Updating the mobile branch is the top integration task.

3. Technology Choices and Rationale

Technology	Why we picked it
ESP32 / PlatformIO	One cheap chip drives every sensor and actuator and has Wi-Fi built in.
MQTT / EMQX	Pub/sub decouples devices, server and app; EMQX serves TCP and WebSocket from one broker.
Flask + Paho (Python)	A small server is enough for the rules engine, live log stream (SSE) and dashboard.
React Native / Expo	One codebase for both phones; MQTT over WebSocket connects straight to the broker.
Docker Compose	Broker and server start with a single command on any laptop.
Storage	Rules persist in `rules.json`; recent events stay in a RAM buffer of 300.

4. Bill of Materials

Component / set	Availability / budget (VND)
ESP32 WT-01 (Ethernet and Wi-Fi)	Available
USB–TTL CH340 adapter	Available
MB-102 breadboard	Available
SG90 servo	46,116
RC522 RFID module	Available
5V relay	Available
HC-SR501 PIR sensor	34,560
LM2596 buck module	45,144
Jumper wires (three types)	Available
12V 2A adapter	46,116
DC power jack	11,124
Budget for remaining items	**183,060**

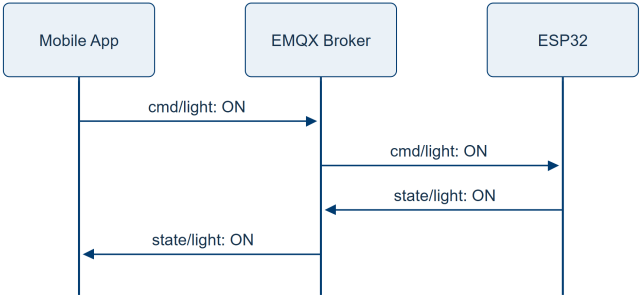
Most parts were already owned; the remaining spend is 183,060 VND (about 7 USD) — a fraction of a commercial smart-home kit.

5. End-to-end Data Flow



PIR and RFID events travel from the ESP32 through the broker into the server's log store and appear on the dashboard in real time; commands take the same path in reverse. The rules engine evaluates every message on the way. Next step: record a live sensor-to-screen demo on the physical model.

5. Light Command and State Feedback



After the refactor there is one command topic in and one log stream back, so the dashboard always shows what the firmware actually did.

Hardware Demo

Live on the model: PIR-triggered light, RFID door opening and servo actuation, with the dashboard reacting in real time.

App Demo

Live on screen: sending a command, seeing the state change, scrolling the event log and editing a rule.

6. Team and Repository

Member	Name	Student ID	Primary ownership
M1	Trần Sỹ Nguyên	20235985	Wi-Fi/MQTT core and firmware interfaces
M2	Lăng Hồng Nguyệt Anh	20235885	PIR, RFID authorization and automation
M3	Lương Ngọc Vũ Long	20235967	Power, wiring, actuators and physical model
M4	Hà Đức Dương	20235923	Mobile dashboard, MQTT control and event UI
M5	Nguyễn Vũ Anh Khoa	20235957	Broker, integration, QA and report coordination

github.com/itstheblank/esp32_smart_home

6. Top Problems Right Now

1. **Mobile contract drift.** The app branch still publishes the old text commands and listens to deprecated topics; it must be migrated to `smarthome/cmd` / `smarthome/logs` and retested end-to-end.
2. **No recorded proof of the full loop yet.** The path works in code, but we still need a live sensor-to-screen recording on the physical model for the demo.
3. **Event history is volatile.** Logs live in a RAM buffer of 300 entries; we have to decide whether checkpoint 2 needs database-backed storage.

Thank You!

Questions and Discussion